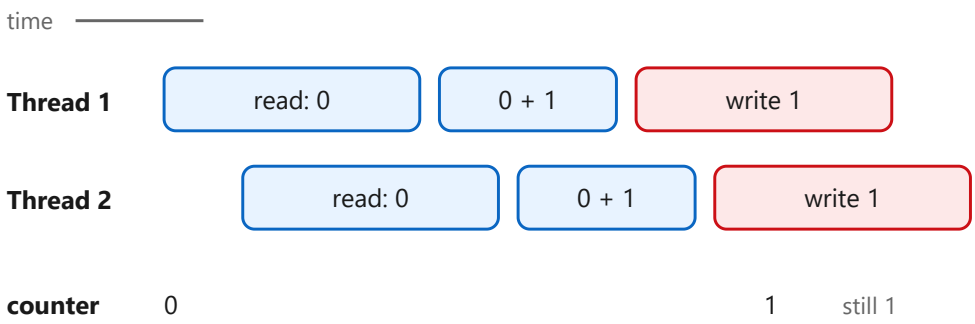
Shared Data and Data Races

Why `++counter` is three steps, what a data race really is, how races break invariants, and the three ways to be safe. Assumes Threads.

Shared data and data races



Expected 2, got 1: both threads read 0 before either wrote. One update is lost.

A data race: the two read-modify-write sequences overlap. ++counter is three steps, not one.

Two threads each increment a shared counter:

```
int counter = 0;
// two threads each run this: expect counter == 4
void work() { for (int i = 0; i < 2; ++i) ++counter; }
```

`++counter` is **three** CPU steps:

1. **load** the value from memory into a register: `r = counter`
2. **add** one to the register: `r = r + 1`
3. **store** the register back to memory: `counter = r`

If B loads between A's load and store, **one increment is lost**. Play the scheduler with **Step A** and **Step B**:

Hint: A loads, B runs fully, A finishes. Or press **Run 500 random schedules**.

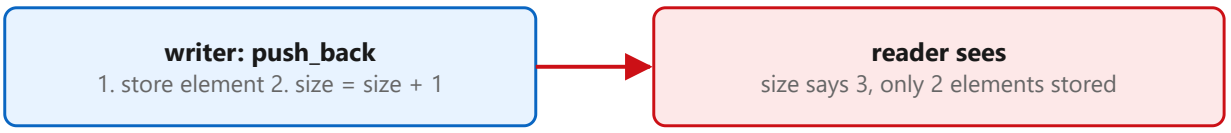
CAVEAT The simulator switches far more often than a real OS. A real race may fail once in thousands of runs, which is why it survives testing.

DATA RACE = UNDEFINED BEHAVIOR Two threads access the same memory, at least one writes, no synchronization. A **race condition** is the wider timing-dependent logic bug.

Why it hurts: shared data has an **invariant** (links agree, count matches items). An update breaks it briefly, and a thread looking then sees corruption. Trouble starts when *two or more* pieces of data change together.

Races break invariants

A single lost increment is the easy case. The real damage comes from **invariants**: facts that must always hold across several variables, such as "the size matches the elements" or "the previous and next pointers agree". Any update that changes two of them takes more than one step, and in the middle the invariant is false.



Invariant: size equals the number of stored elements.

Between step 1 and step 2 it is false. Another thread reading now sees garbage.

A mutex makes the two steps look like one.

An invariant spans several variables, so a write sequence is briefly invalid.

TERM	MEANING
data race	two threads access the same memory, at least one writes, with no synchronization: undefined behavior
race condition	the result depends on the timing of threads; wider than a data race and possible even with atomics or locks
invariant	a condition on shared data that must hold between operations
deadlock	threads wait for each other forever (next lessons)

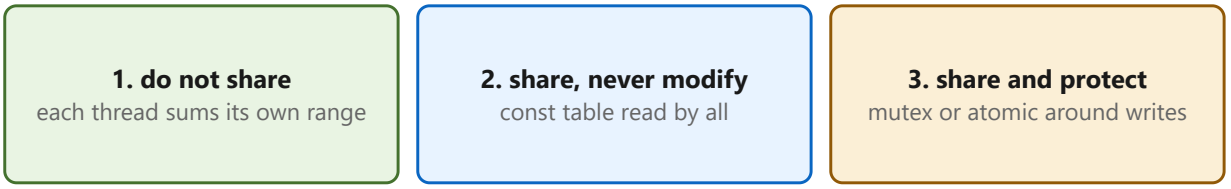
UNDEFINED BEHAVIOR, NOT STALE DATA A data race is not just "you might read an old value". The compiler assumes there are none, and may reorder or remove code on that assumption.

Three ways to be safe

STRATEGY	HOW
1. Do not share	own data per thread (locals, `thread_local`), merge afterwards. Best option.
2. Share, never modify	`const` data is safe to read from many threads.
3. Share and protect	a **mutex** or an **atomic**.

The three strategies in code

Here is what each strategy looks like for the same job: add up a large array with four threads.



Each step costs more than the one before: sharing needs synchronization, and synchronization needs waiting. Prefer 1, then 2, and use 3 only for data that really must be shared.

The cheapest safe design is the one that never shares.

```cpp
// 1. do not share: each thread sums its own slice into its own variable,
// then we merge
// one slot per thread: no two threads write the same slot
long long part[4] = {};
for (int t = 0; t < 4; ++t)
    workers.emplace_back([&, t] { for (size_t i = t; i < v.size(); i += 4)
part[t] += v[i]; });
for (auto& w : workers) w.join();
// join makes every write visible here
long long total = part[0] + part[1] + part[2] + part[3];
```

STRATEGY	COST	WATCH OUT FOR
do not share	none while running; a merge at the end	neighbouring slots on one cache line slow each other (false sharing): pad them if it matters
share, never modify	none	the data must really be `const` for the whole time
share and protect	waiting on the lock or the atomic	the invariant, the lock order, and holding the lock briefly

The thread that calls `join()` sees everything the joined thread wrote: `join` is itself a synchronization point.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Shared data and data races	C++ Concurrency in Action	3.1 Problems with sharing data between threads (invariants, race conditions)	60 to 63